

**Design suitable data structures and implement Pass-I of a two-pass assembler for pseudo-machine.**

|      |       |       |       |
|------|-------|-------|-------|
|      | START | 201   |       |
|      | MOVER | AREG, | = '5' |
|      | MOVEM | AREG, | X     |
|      | MOVER | BREG, | = '2' |
|      | LTORG |       |       |
| NEXT | ADD   | AREG, | = '1' |
|      | SUB   | BREG, | = '2' |
|      | BC    | LT,   | NEXT  |
|      | MULT  | CREG, | = '4' |
|      | STOP  |       |       |
| X    | DS    | 1     |       |
|      | END   |       |       |

**Design suitable data structures and implement Pass-II of a two-pass assembler for pseudo-machine.**

|      |        |        |       |
|------|--------|--------|-------|
|      | START  | 201    |       |
|      | MOVER  | AREG,  | = '5' |
|      | MOVEM  | AREG,  | X     |
| L1   | MOVER  | BREG,  | = '2' |
|      | ORIGIN | L1+3   |       |
|      | LTORG  |        |       |
| NEXT | ADD    | AREG,  | = '1' |
|      | SUB    | BREG,  | = '2' |
|      | BC     | LT,    | BACK  |
|      | LTORG  |        |       |
| BACK | EQU    | L1     |       |
|      | ORIGIN | NEXT+5 |       |
|      | MULT   | CREG,  | = '4' |
|      | STOP   |        |       |
| X    | DS     | 1      |       |
|      | END    |        |       |

**Design suitable data structures and implement Pass-I of a two-pass assembler for pseudo-machine.**

|      |        |        |    |
|------|--------|--------|----|
|      | START  | 100    |    |
| L1   | MOVER  | AREG,  | X  |
|      | MOVEM  | AREG,  | Y  |
|      | ORIGIN | L1+3   |    |
| NEXT | ADD    | AREG,  | X  |
|      | SUB    | BREG,  | Y  |
|      | BC     | LT,    | L1 |
|      | ORIGIN | NEXT+5 |    |
|      | MULT   | CREG,  | Z  |
|      | STOP   |        |    |
| X    | DS     | 2      |    |
| Y    | DS     | 1      |    |
| Z    | DC     | '9'    |    |
|      | END    |        |    |

**Design suitable data structures and implement Pass-I of a two-pass macro- processor.**

|   |       |                 |
|---|-------|-----------------|
|   | MACRO |                 |
|   | ONE   | &O, &N, &E=AREG |
|   | MOVER | &E, &O          |
|   | ADD   | &E, &N          |
|   | MOVEM | &E, &O          |
|   | MEND  |                 |
|   | MACRO |                 |
|   | TWO   | &T, &W, &O=DREG |
|   | MOVER | &O, &T          |
|   | ADD   | &O, &W          |
|   | MOVEM | &O, &T          |
|   | MEND  |                 |
|   | START |                 |
|   | READ  | O               |
|   | READ  | T               |
|   | ONE   | O, 9            |
|   | TWO   | T, 7            |
|   | STOP  |                 |
| O | DS    | 1               |
| T | DS    | 1               |
|   | END   |                 |



**Design suitable data structures and implement Pass-II of a two-pass macro- processor.**

```
MACRO
ONE      &O, &N, &E=AREG
MOVER    &E, &O
ADD      &E, &N
MOVEM    &E, &O
MEND
MACRO
TWO      &T, &W, &O=DREG
MOVER    &O, &T
ADD      &O, &W
MOVEM    &O, &T
ONE      O, 9, &E=CREG
MEND
START
READ     O
READ     T
TWO      T, 7
PRINT    O
PRINT    T
STOP
O  DS   1
T  DS   1
END
```

Write a program to solve Classical Problems of Synchronization using Mutex and Semaphore.

[Producer-Consumer Problem/ Readers and Writers Problem/ Dining-Philosopher Problem/ Sleeping Barber Problem]

**OR**

Write a program to simulate CPU Scheduling Algorithms.

- (i) FCFS,
- (ii) Round Robin

Write a program to solve Classical Problems of Synchronization using Mutex and Semaphore.

[Producer-Consumer Problem/ Readers and Writers Problem/ Dining-Philosopher Problem/ Sleeping Barber Problem]

**OR**

Write a program to simulate CPU Scheduling Algorithms.

- (i) Priority (Non-preemptive),
- (ii) SJF (Preemptive)



Write a program to simulate Memory placement strategies,

- (i) best fit,
- (ii) first fit,
- (iii) next fit

**OR**

Write a program to simulate Page replacement algorithms,

- (i) FIFO
- (ii) OPTIMAL

Write a program to simulate Memory placement strategies,

- (i) best fit,
- (ii) first fit,
- (iii) worst fit

**OR**

Write a program to simulate Page replacement algorithms,

- (i) LRU
- (ii) FIFO

**ELECTIVE:**

**[IoT & ES]** Understanding the connectivity of Raspberry-Pi with IR sensor. Write an application to detect obstacle and notify user using LEDs.

**[DS]** Implementation of Inter-process communication using socket programming:  
implementing multithreaded echo server.

**OR**

**[DS]** Implementation of RPC Mechanism.

**[HCI]**

Design a User Interface in Python.

**ELECTIVE:**

**[IoT & ES]** Understanding the connectivity of Raspberry-Pi circuit with temperature sensor. Write an application to read the environment temperature. If temperature crosses a threshold value, generate alerts using LEDs.

**[DS]** Simulation of election algorithms (Ring and Bully).

**OR**

**[DS]** Implementation of Clock Synchronization:

- (i) NTP
- (ii) Lamport's clock

**[HCI]**

Design a User Interface in Python.